

# ARDUINO UNO R4-WIFI - ARDUINO PROGRAMMING BASICS

*Abdul Kadir (Primasoft RG, 2024)*

## Abstract

This Article is intended to introduce Arduino programming basics so the readers can be familiar with the basics of programming before exploring any hardware further. The topics discussed in this chapter::

- Framework of sketches
- Identifiers
- Data types
- Variables and literals
- Constants
- Functions
- Objects

## Understanding the Basic of Arduino Sketches

So far, you have written several sketches to make various projects and you are familiar with `setup()` and `loop()` functions. You are also familiar with `const` statements to declare constants that laid

outside those two functions. All code located outside and before `setup()` and `loop()` definitions will be processed first. The code contains preprocessor directives such as `#define` or statements such as `const` statements.

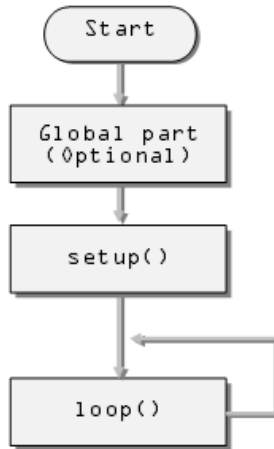
The function `setup()` will be run once at first after the code before this function is processed. Therefore, this function is useful for initialization. Then, after the `setup()` is, the function `loop()` is executed repeatedly.

*Note:*

A statement is an instruction in a sketch. You can imagine a statement like a sentence in the human language. A statement in Arduino language usually ends with a semicolon, like a sentence that must end with a period.

The function `loop()` is used to repeat something. In the sketch `first.ino` (Chapter 1), no statement is provided. Therefore, no repetition is accomplished. However, although no process is to be repeated in the `loop()`, this function must be written explicitly.

Figure 3.1 shows the pattern of each sketch, including the `setup()` and `loop()` functions. Based on that figure, the `loop()` function is always repeated.



**Figure 3.1** Sketch execution in Arduino

The global part is usually used to give preprocessor directives such as `#include` and `#define`, creating objects, declaring constants, or declaring global variables. This part is optional. The previous sketch has no global part. However, several examples discussed later will contain this part.

## How to Create Identifiers

Identifiers are often used in sketches. Identifiers are names created by programmers to define variables or other elements in Arduino sketches, such as functions and constants. The programmers have the freedom to give a name as long as they do not use any Arduino keywords or special names used by the compilers, such as `for` and `bool`, and obey the rules of identifiers.

The rules for defining identifiers:

- The identifier must start with either a letter or an underscore symbol (`_`).
- The next characters can be a letter, a digit, or an underscore symbol (`_`).

- Lowercase letters and uppercase letters are considered different (case sensitive). So, resistor and Resistor are different identifiers.

Table 3.1 shows several valid and invalid identifiers.

**Table 3.1** Examples of several valid and invalid identifiers

| Valid Identifier                | Invalid Identifier                                                                |
|---------------------------------|-----------------------------------------------------------------------------------|
| <code>doIt</code>               | <code>do</code> (It is a keyword)                                                 |
| <code>two_multiply_three</code> | <code>two*three</code> (cannot contain the * symbol)                              |
| <code>priceInDollar</code>      | <code>\$price</code> (cannot contain the \$ symbol)                               |
| <code>pin13</code>              | <code>13pin</code> (The first character must be a letter or an underscore symbol) |
| <code>sensorValue</code>        | <code>sensor Value</code> (cannot contain space characters)                       |
| <code>LED_PIN</code>            | <code>LED-PIN</code> (The – symbol cannot be used)                                |

*Note:*

- Identifiers that start with a lowercase letter are usually used for naming variables. Whilst, identifiers using all uppercase letters are commonly used for declaring constants.
- When creating a variable name containing more than one word, the two words should be separated by an underscore. Another approach is by using camel notation, by mixing lowercase and uppercase letters in which all the beginning letters of each word except the first word are written using uppercase letters.
- An example of using an underscore symbol to separate two words: `temp_in_room`
- An example of using the camel model:

## Understanding Data Types

Data types determine how data is stored in memories and also to be processed in an expression. Therefore, understanding the data types supported by Arduino is very important. Table 3.2 shows simple data types in Arduino programming.

**Table 3.2** Simple data types

| Data Type               | Storage (bytes) | Description                                                                                                           |
|-------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------|
| byte                    | 1               | Integer: between 0 and 255                                                                                            |
| short<br>(or short int) | 2               | Integer: between -32768 and +32767                                                                                    |
| int                     | 2               | Same as short                                                                                                         |
| unsigned int            | 2               | Positive integer: between 0 dan 65535                                                                                 |
| word                    | 2               | Same as unsigned int                                                                                                  |
| long<br>(or long int)   | 4               | Integer: between -2147483648 and +2147483647                                                                          |
| unsigned long           | 4               | Positive integer: between - 2147483648 and 2147483647                                                                 |
| float                   | 4               | Floating-point numbers with a precision of 6-7 decimal digits. Its number is between 3.4028235E+38 and -3.4028235E+38 |
| double                  | 4               | Same as float                                                                                                         |
| bool                    | 1               | Boolean type, with the possibility value is true or false                                                             |
| char                    | 1               | This type stores one of the ASCII characters                                                                          |
| void                    | 0               | Only used for defining functions and                                                                                  |

| Data Type | Storage (bytes) | Description        |
|-----------|-----------------|--------------------|
|           |                 | declaring pointers |

Arduino supports a special operator named `sizeof`, which is useful for getting the of size a data type. For example, `size(int)` gives 4.

## Variables and Literals

When we talk about data, principally it can be differentiated into two categories, variables and literals. Variables are names for memory locations used in a sketch to store data and its data can be changed at any time when the sketch runs. Literals represent fixed values that do not change throughout the sketch.

### Declaring and Assigning Variables

Before using a variable, the variable must be declared first. The intention of declaring variables is to introduce the names of variables and define their data types. The syntax of a variable declaration is as follows:

```
data_type list_of_variables;
```

The data types such as `int`, `boolean`, or others as listed in Table 3.2 can be used. After the data type, a variable or several variables can be placed. In this case, if there are two variables or more, a comma (and optionally a space) must be inserted between two variable names. For examples:

- ❶ `int result;`
- ❷ `char aLowercase;`
- ❸ `bool found;`
- ❹ `float tempInCelcius, tempInFahrenheit;`

Numbers ❶ , ❷ , and ❸ are examples of declaring a variable in each statement. The result variable is an `int` variable, which can store an integer number between -32768 and +32767. The `aLowercase` variable is a `char` variable, which can store any single character (not only a lowercase letter). The `found` variable is a `bool` variable, so the value `true` or `false` can be stored in this variable. Meanwhile, the number ❹ gives an example of declaring two variables in one statement. At least, a comma must separate the variables `tempInCelcius` and `tempInFahrenheit`. A space after the comma is optional, but it is better to insert it for readability.

After declaring a variable, no guarantee that its value is zero. Therefore, make sure that there is a statement for assigning its value after declaring a variable. To assign a value to a variable, an assignment operator can be used. The syntax for assigning a value to a variable is as follows:

```
variable_name = value;
```

The value can be a literal, a function, or an expression. Several examples of variable assignments are as follows:

- `result = 0; // Assigning a number literal`
- `aLowercase = 'a'; // Assigning a character literal`
- `found = true; // Assigning a Boolean literal`
- `tempInCelcius = 29.2; // a floating-point number`
- `result = 2 * 3; // Multiplication 2 and 3`
- `result = sqrt(25); // a return value of function call`

It is possible to give a different type of value to a variable. Although the result variable is an `int` variable, the following assignment is valid:

```
result = 2.3;
```

In this case, the integer part of 2.3 is stored in the result variable and the fractional part is ignored.

Assigning a value to a variable can be done when declaring the variable. The term initialization is used in this case. The syntax of variable initialization is as follows:

```
data_type variable_name = value;
```

For example, the following statement is for declaring variable `total` as an `int` variable and storing 0 to that variable:

```
int total = 0;
```

That statement is equivalent to the following two statements:

```
int total;
```

```
total = 0;
```

The following sketch shows how to declare a variable in the `loop()` function and how to use the variable:



## Try Yourself!

Sketch: local.ino

```
// Declaring a variable in loop()

void setup() {
  Serial.begin(115200);
}

void loop() {
  int number = 0;

  number++; // Increase the content of the number by 1

  Serial.print("number = ");
  Serial.println(number);

  delay(1000); // Delay 1 second
}
```

## End of Sketch

The outputs of this sketch on the Serial Monitor:

```
number = 1  
number = 1  
number = 1  
...
```

Figure 3.3 shows that the line with “number = 1” is repeated. The value of 1 comes from the `number` variable. In the `loop()` function, the following statement declares the `number` variable as an `int` variable assigning zero to the variable:

```
int number = 0;
```

Then, the following statement makes the content of the `number` is 1 since the `++` operator increases the content of the `number` by 1:

```
number++;
```

Therefore, the following statement displays 1 in the Serial Monitor:

```
Serial.println(number);
```

A variable such as `number` in the previous example is known as a local variable in the `loop()` function. The variable will be declared when `loop()` is run and will disappear when the function is terminated. Therefore, the output of `Serial.println(number);` is always 1.

We will get different results when we use a global variable. Such variables will be recognized by the `loop()` function (and other functions, such as `setup()`). In this case, the variable should be declared in the global part (see Figure 3.1). To see the effect of a variable global, you can try the following sketch:



## Try Yourself!

Sketch: global.ino

```
// Declare a global variable

int number = 0; // Global variable

void setup() {
  Serial.begin(115200);
}

void loop() {
  number++; // Increase the the  number by 1

  Serial.print("number = ");
  Serial.println(number);

  delay(1000);
}
```

**End of Sketch**

This sketch is based on the variable sketch with a little modification. The statement

```
int number = 0;
```

in the sketch now is located before the definition of the `setup()` function in the global sketch. Therefore, the variable `number` in the sketch is a global variable. The following outputs on the Serial Monitor shows the effect of that variable:

```
number = 1
number = 2
number = 3
...
```

You can see that now the value of the variable `number` is increased for every calling `loop()` by the system.

## How to Write Literals

Several data types have been introduced. Now, the focus is on how to write literals for various data types in sketches.

### Integer Data Types

Integer literals are written without fractional values. By default, an integer literal is an int. Special suffixes are provided to specify certain integer types:

- A suffix 'u' or 'U' is used to represent an unsigned data format. Example: 61234u.
- A suffix 'l' (lowercase of L) or 'L' is used to represent a long data format. Example: 456789124L.
- A suffix 'uL' or 'UL' is used to represent an unsigned long data format. Example: 4000000000UL.

Besides using a decimal system (base 10), integer literals can be represented in binary (base 2), octal (base 8), or hexadecimal (base 16) systems. Table 3.3 explains and gives examples for all bases.

**Table 3.3** Integer literals in various bases

| Base         | Description                                                                                              | Examples                                                                                                                        |
|--------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Decimal (10) | Except for 0, any integer literal should not be started with zero.<br>All digits recognized: 0 to 9      | <ul style="list-style-type: none"><li>• 20</li><li>• 255</li></ul>                                                              |
| Binary (2)   | Any binary literal is started by B. The maximum number of digits is 8.<br>All digits recognized: 0 and 1 | <ul style="list-style-type: none"><li>• B10100<br/>(same as 20 decimal)</li><li>• B11111111<br/>(Same as 255 decimal)</li></ul> |
| Octal (8)    | Any octal literal is started by 0                                                                        | <ul style="list-style-type: none"><li>• 024 (same as</li></ul>                                                                  |

| Base             | Description                                                                                                              | Examples                                                                                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | (zero). Digits recognized: 0 to 7                                                                                        | 20 decimal) <ul style="list-style-type: none"> <li>• 0377 (Same as 255 decimal)</li> </ul>                                                                                        |
| Hexadecimal (16) | Any hexadecimal literal is started by 0x (zero and x or zero and X).<br>Characters recognized: 0 to 9, A to F, or a to f | <ul style="list-style-type: none"> <li>• 0x14 (same as 20 decimal)</li> <li>• 0xFF (same as 255 decimal)</li> <li>• 0xff (same as 0xFF)</li> <li>• 0XFF (same as 0xFF)</li> </ul> |

By using `Serial.println()`, an integer value can be displayed in a decimal, octal, hexadecimal, or binary format. In this case, the second argument of `println()` must be supplied. The choice for the second argument is BIN, HEX, OCT, or DEC. The following sketch gives several examples of `Serial.println()` to show the effect of the second argument of `println()`.



## Try Yourself!

Sketch: format.ino

```
// Examples of how to display a number
//   in decimal, binary, octal, and hexadecimal

void setup() {
  Serial.begin(115200);

  int number = 213;

  Serial.print("Without second argument: ");
  Serial.println(number);
```

```
Serial.print("Using DEC: ");  
Serial.println(number, DEC);  
  
Serial.print("Using BIN: ");  
Serial.println(number, BIN);  
  
Serial.print("Using OCT: ");  
Serial.println(number, OCT);  
  
Serial.print("Using HEX: ");  
Serial.println(number, HEX);  
}  
  
void loop() {  
}
```

### End of Sketch

The output of this sketch:

```
Without second argument: 213  
Using DEC: 213  
Using BIN: 11010101  
Using OCT: 325  
Using HEX: D5
```

## Floating-Point Data Types

A floating-point number is a real number. The number has a fractional part. Several examples of floating-point literals are as follows:

- 123.45
- 0.05
- .05 (same as 0.05)
- 2.15e4 or 2.15E4 (same as  $2.1 \times 10^4$ )
- 2.15e-4 (same as  $2.1 \times 10^{-4}$ )

## Boolean Data Type

Two literals can be used to assign a Boolean variable: 1) `true` and 2) `false`. Boolean data is often used in decision-making or repetition. Here is an example of using a Boolean variable:



## Try Yourself!

Sketch: bool.ino

```
// Example of how to use a Boolean variable

void setup() {
  Serial.begin(115200);

  boolean found = false;

  Serial.print("1. found: ");
  Serial.println(found);

  // Change to true
  found = true;

  Serial.print("2. found: ");
  Serial.println(found);
}

void loop() {
}
```

**End of Sketch**

The output this sketch:

```
1. found: 0
2. found: 1
```

The value of `false` is displayed as 0 and The value of `true` is displayed as 1.

### Character Data Type

The `char` type is related to an ASCII character. ASCII stands for American Standard Code for Information Interchange. An ASCII code is the numerical representation of a character such as 'A' or '@'. You can visit <http://www.asciitable.com> to see the complete ASCII table.

A char literal is written by quoting the characters. Here is an example of a char literal:

`'A'`

In this case, a pair of single quotes (') encloses the character.

Here is an example of declaring a char variable and how to assign its value:



## Try Yourself!

Sketch: char.ino

```
// Example of how to use a char variable

void setup() {
  Serial.begin(9600);

  char ch = 'A';

  Serial.print("1. ch : ");
  Serial.println(ch);

  // Try to add the content of ch by 1
  ch = ch + 32;

  Serial.print("2. ch : ");
  Serial.println(ch);

  // Try to add the content of ch by 32
  ch = ch + 1;

  Serial.print("3. ch : ");
  Serial.println(ch);

  // Display the ASCII code
  Serial.print("4. The ASCII Value of ");
  Serial.print(ch);
  Serial.print(" is ");
  Serial.println(ch, DEC);
}

void loop() {
}
```

## End of Sketch

The output this sketch:

```
1. ch: A
2. ch: a
3. ch: b
4. The ASCII value of b is 98
```

Arduino supports the character escape sequences as shown in Table 3.5. These sequences are useful for writing special characters, including for writing a single quote. For example, to represent a single quote, you must write `"\'"`.

**Table 3.5** Character escape sequences

| Character Escape Sequences | Meaning                                                       |
|----------------------------|---------------------------------------------------------------|
| <code>\0</code>            | The NULL character                                            |
| <code>\f</code>            | Form feed                                                     |
| <code>\n</code>            | Newline                                                       |
| <code>\r</code>            | Carriage return                                               |
| <code>\t</code>            | Horizontal tab                                                |
| <code>\v</code>            | Vertical tab                                                  |
| <code>\\</code>            | Backslash ( <code>\</code> )                                  |
| <code>\'</code>            | Single quote ( <code>'</code> )                               |
| <code>\"</code>            | Double quote ( <code>"</code> )                               |
| <code>\?</code>            | The <code>?</code> symbol                                     |
| <code>\ooo</code>          | A character in which its octal is ooo (three octal digits)    |
| <code>\xhh</code>          | A character in which its octal is hh (two hexadecimal digits) |

## String Data Types

There are two kinds of string data types in Arduino: 1) null terminated string and 2) String data type. Both strings will be discussed later in Chapter 6. However, the two kinds of strings use the same rule for

writing string literals. All string literals are written by quoting the string using double quotes. Here are five examples of string literals:

- "Test..test..123!" (String that combines digits, letters, and symbols)
- "123" (String that contains three digits)
- "A" (String that contains a letter)
- " " (String that contains one space character)
- "" (Empty string; no character at all)

A string literal can also contain special characters as shown in Table 3.5. For example, "123\n456" is a string literal that contains a newline between characters 3 and 4.

## How to Write Constants

A constant is a value that never changes with a given name. A constant can be declared by using `const` or `#define`.

### Writing a Constant using `const`

A constant is usually used when a literal appears more than once in a sketch. Another reason is for clarity purposes. The syntax for defining a constant is as follows:

```
const data_type constant_name = value;
```

A constant is declared by using the `const` keyword. For example, the following statement declares `LED_PIN` as an `int` and its value is 13:

```
const int LED_PIN = 13;
```

## Writing a Constant Using #define

Besides using `const`, a constant can be defined by using a preprocessor directive `#define`. For example, look at the following statement:

```
const byte LED_PIN = 13;
```

This statement can be written as follows:

```
#define LED_PIN 13
```

Although it seems that using `#define` is simpler, it is better to use `const`, instead of `#define`. By using `const`, there is a type checking when the sketch is compiled.

## Understanding Expression and Operators

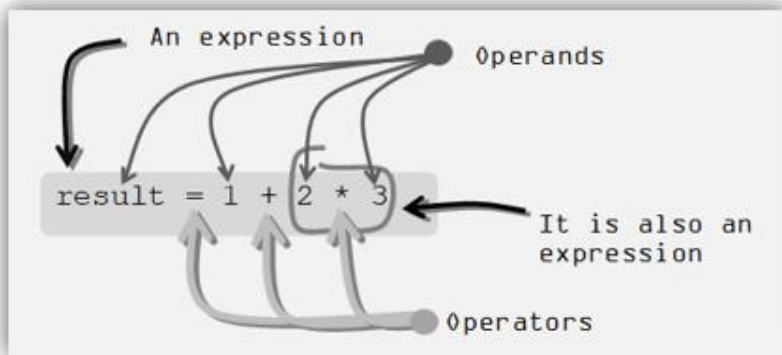
An expression in Arduino programming usually involves operands and operators. For example,

```
result = 0;
```

In this case, `result` and `0` are operands and `=` is an operator. Another expression is as follows:

```
result = 1 + 2 * 3;
```

Figure 3.9 illustrates the expression, operands, and operators.



**Figure 3.9** Illustration of an expression

The previous example shows an expression using arithmetic operators. However, expressions are not related to arithmetic operations only. Later, you will learn string expressions.

## Arithmetic Operators

In Arduino, there are several arithmetic operators as listed in Table 3.7. The +, -, / and \* can be used for integer and floating-point data types. If all operands are integers, the result is also an integer value. If there is at least one operand with a floating-point value, the result is a floating-point number. When the / operator is used for integer data types, the result is an integer value. Any fractional part is truncated.

**Table 3.7** Arithmetic operators

| Operator | Description      | Examples                                                                                                                                                                        |
|----------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| *        | Multiplication   | <ul style="list-style-type: none"> <li>• <math>2 * 3 \rightarrow 6</math></li> <li>• <math>2 * 3.0 \rightarrow 6.0</math></li> </ul>                                            |
| /        | Division         | <ul style="list-style-type: none"> <li>• <math>3 / 2 \rightarrow 1</math></li> <li>• <math>3 / 2.0 \rightarrow 1.5</math></li> </ul>                                            |
| %        | Modulus operator | <ul style="list-style-type: none"> <li>• <math>8 \% 2 \rightarrow 0</math></li> <li>• <math>8 \% 3 \rightarrow 2</math></li> <li>• <math>7 \% 2 \rightarrow 1</math></li> </ul> |

| Operator | Description | Examples                                                                                                                             |
|----------|-------------|--------------------------------------------------------------------------------------------------------------------------------------|
| +        | Addition    | <ul style="list-style-type: none"> <li>• <math>1 + 2 \rightarrow 3</math></li> <li>• <math>1 + 2.5 \rightarrow 3.5</math></li> </ul> |
| -        | Subtraction | <ul style="list-style-type: none"> <li>• <math>5 - 2 \rightarrow 3</math></li> <li>• <math>5 - 2.0 \rightarrow 3.0</math></li> </ul> |

The following sketch gives an illustration of how to get the correct answer when we want to convert a temperature in Celsius to Reaumur:



## Try Yourself!

Sketch: temperature.ino

```
// Example of how to write a correct expression
//   for converting temperature in Celsius to
//   Fahrenheit by using the formula:
//
//   Reaumur = 4 / 5 * Celsius

void setup() {
  float celsius = 32;
  float reaumur;

  Serial.begin(115200);

  // First expression
  reaumur = 4 / 5 * celsius;
  Serial.println(reaumur);

  // Second expression
  reaumur = 4.0 / 5 * celsius;
  Serial.println(reaumur);
}

void loop() {
}
```

End of Sketch

The result of the temperature sketch is shown as below:

```
0.00  
25.60
```

You can see that both displayed values are different from the manual calculation.

Why the result of the first expression is zero? Let's try to analyze the code in the temperature sketch. First, see the following declarations:

- `float celsius = 32;`
- `float reamur;`

The first statement declares a variable named `celsius` as a float variable and the variable is initialized by 32. The second statement just declares a variable named `reamur` as a float variable. Then, the following statement converts the value of `celsius` into `reamur`:

```
reamur = 4.0 / 5 * celsius;
```

But, the statement gives 0 to `reamur`. It is caused by the expression `4 / 5`. That expression always yields 0 because the division between an integer and an integer will give an integer. To correct the conversion result from Celsius to Reaumur, the expression `4 / 5` should be written as `4.0 / 5`, `4 / 5.0`, `4.0 / 5.0`, or `0.8`. Therefore, the statement the following statement produces a correct result:

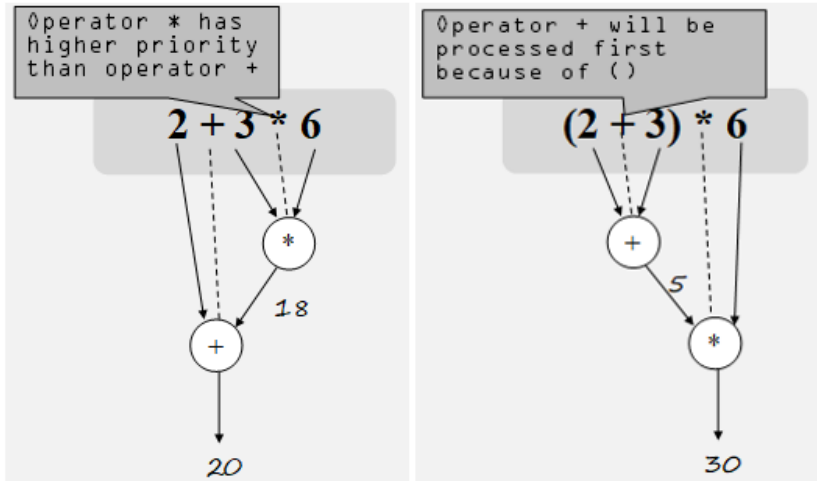
```
reamur = 4.0 / 5 * celcius;
```

## Operator Precedence

Arduino uses the precedence rules of the operators when evaluating an expression. The precedence rules can be used to explain why an expression such as `2 + 3 * 6` will result in 20, not 30. Please visit the following website to know the precedence of all operators:

[https://en.cppreference.com/w/cpp/language/operator\\_precedence](https://en.cppreference.com/w/cpp/language/operator_precedence)

Based on the precedence of operators, it is clear that  $2 + 3 * 6$  will result in 20, as shown in Figure 3.11 (left), since the precedence of  $*$  is higher than the precedence of  $+$ . However,  $(2 + 3) * 6$  will give 30, as shown in Figure 3.11 (right). In other words, a pair of  $()$  can be used to change the priority in an expression evaluation.



**Figure 3.11** How to evaluate  $2 + 3 * 6$  and  $(2 + 3) * 6$

## Type Conversion

Type conversion denotes how to change a value in a certain data type to another. In Arduino programming, type conversion can be done either implicitly or explicitly. For example, the num variable is an int variable. When there is a statement as follows, a type conversion will be done implicitly:

```
num = 20.5;
```

In this case, the floating-point number 20.5 will be converted to int by ignoring the fractional part. Then, the number 20 will be assigned to variable num.

Now, suppose sum and num are int variables. The content of sum is 24 and the content of num is 5. Then, there is a statement as follows:

```
float average = sum / num;
```

What is the value stored in the average variable? The answer is 4.0, not 4.8, because the integer division between sum and num produces an integer number. So, how to get a value of 4.8 in the variable average? The solution is using type cast or type conversion explicitly. The statement needed is as follows:

```
float average = float(sum) / num;
```

or

```
float average = sum / float(num);
```

The format like `float(sum)` is called a type cast. Table 3.13 lists all type casts in Arduino.

**Table 3.13** Type casts

| Type Cast | Description                                       |
|-----------|---------------------------------------------------|
| char()    | Returns a char value based on the argument value  |
| byte()    | Returns a byte value based on the argument value  |
| int()     | Returns an int value based on the argument value  |
| word()    | Returns a word value based on the argument value  |
| long()    | Returns a long value based on the argument value  |
| float()   | Returns a float value based on the argument value |

## Creating Functions

Sometimes, we need to create functions beside `setup()` and `loop()` for any reason, to simplify the code or to decrease the memory needs. Figure 3.22 illustrates the flow of execution when a function is called. The numbers 1, 2, and 3 show the path when `blink(2, 3)` is called. Whilst, numbers 4, 5, and 6 show the path when `blink(3, 3)` is called.

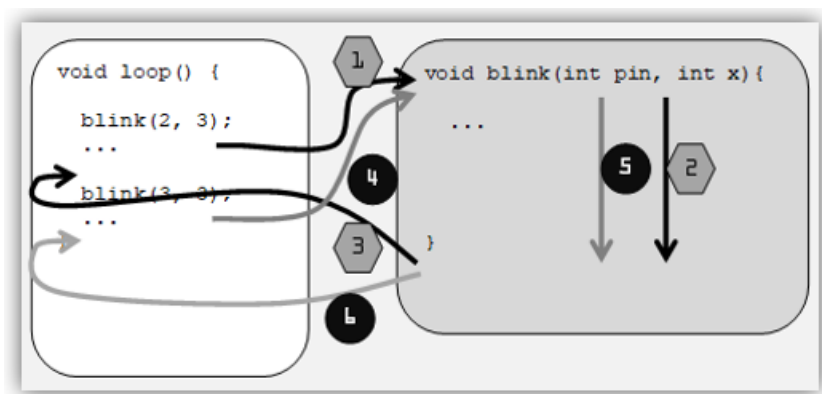


Figure 3.22 Flow of execution of two function calls

## The General Format for Creating Functions

When you write `setup()` and `loop()` in many sketches, you have created functions. The general format of creating a function (or usually called defining a function) is as follows:

```
data_type function_name(type param_1, ..., type param_N) {
    statements;
    return value;
}
```

In this case,

- The *data\_type* in front of the *function\_name* determines the data type of the function return value.
- The following statement terminates the execution of the function and gives value as the return value of the function:

```
return value;
```

This statement does not exist if the type of the return value of the function is `void`.

- The parentheses after the *function\_name* can contain zero, one, or many formal parameters. For example, the `pin` in the

function `blink()` (Figure 3.22) is a formal parameter. Any formal parameter contains its type. For example, the type of `pin` in the `blink()` function is `int`.

Any new function definition can be defined before or after `setup()` and `loop()` definitions.

## Function Calls

If a function has been defined, you can call the function by using one of the following syntaxes.

- For a function with the return type is void:  
`function_name(arg, ...);`
- For a function with the return type is not void:
  - 1) `variable = function_name(arg, ...); variable`
  - 2) `function_name(arg, ...)`

The first example stores the return value to a variable and the second example puts the return value in an expression.

Two following terms should be noted when we talk about functions:

- Formal parameters are parameters that appear in the function header in the definition of the functions.
- Actual parameters (often called arguments) are parameters that appear in the function call.

Figure 3.23 shows the visual difference between a formal parameter and an actual parameter.

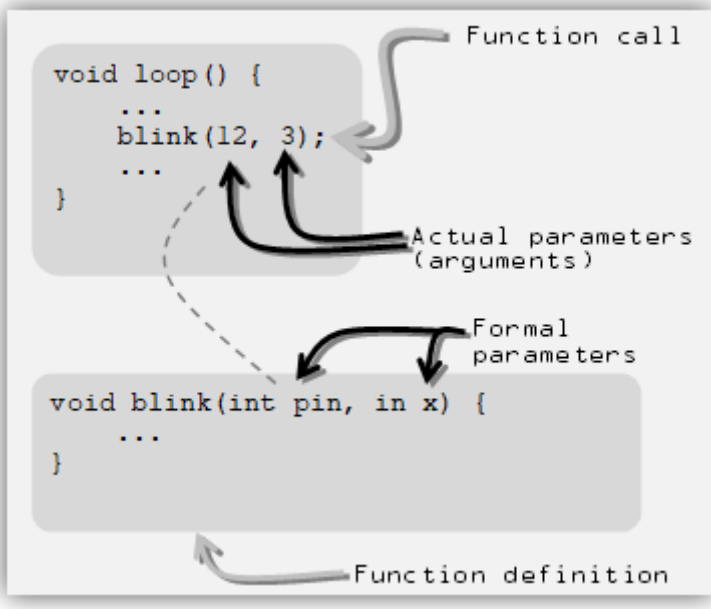


Figure 3.23 An actual and a formal parameter

### Guidance for using Global versus Local Variables

Based on the scope of variables, referring to regions in the sketch where variables can be accessed, variables can be classified into two kinds: 1) local variables and 2) global variables. All variables declared within a function or a block are called local variables. All variables declared outside any function are global variables. Both local variables and global variables have been introduced in Chapter 3. In this section, how to choose the global or local variables will be emphasized.

So, how to choose global or local variables in our sketches? The answer is very simple. Here is the guidance:

- If the value is not needed to be recognized in other functions, use a local variable instead of a global variable.

- If the value needs to be recognized by another function and we can put it as the function argument, put it as a function argument rather than as a global variable.
- Otherwise, put the value in a global variable.

For example, if we want to pass a value from a `loop()` to the next `loop()` calling, a global variable is required. However, if you just read a value from a sensor in the `loop()`, a local variable is enough.

## Learning Objects

Today, object-oriented programming (OOP) is a well-known approach in programming, which combines data and functions into a unit called an object. It is different from traditional programming named structured



**Figure 3.24** An object encapsulates data and functions

programming, which separates data and functions. The focus of structured programming is only on functions, while OOP combines data and functions.

*Note:*

In OOP, functions are known as member functions or methods. Whilst, data is commonly called data members or properties. In Arduino, the member functions are often called just as functions.

Combining data and functions in objects makes OOP powerful in solving problems. It supports reusability, which means that the code can be expanded easily without creating new code from scratch. Based on an existing class, a template of objects, another class can be created easily by using another feature called inheritance. In other

words, a class or more can be implemented by inheriting from an existing class.

Learning objects is very important because many libraries implement classes and you can utilize the libraries by creating objects. Therefore, how to create objects will be discussed in this book. Then, later you will learn how to create a class as well.

A class is usually stored in two separate files. The first file has an extension .h. This file contains the class definition. Hence, sometimes you must include this kind of file. For example, you can insert the following code in a sketch if you want to store or read data into or from EEPROM:

```
#include <EEPROM.h>
```

The second file has an extension .cpp. This kind of file contains the code for implementing the class.

## **Example of An Object**

Serial is an example of a predefined object in Arduino. This object represents a serial port (also known as a UART or USART). This object has several functions (or member functions). The general format of using a function containing in an object is as follows:

*object\_name.function\_name(arguments)*

For example, you are familiar with `Serial.begin()` and `Serial.println()`. Those writings fulfill the above syntax. Meanwhile, the general format of using a data member containing in an object is as follows:

*object\_name.data\_name*

However, usually, data members are private data. So, data cannot be accessed by the object (only in the implementation of the class as explained later).

*Note:*

In the OOP terminology, a class variable is known as a class object or class instance. To simplify, a class object is often called an object.

## **How to Create Objects**

Creating an object is based on a class. The general format for creating an object is as follows:

```
class_name object_name(arguments);
```

For example, String is a predefined class. You can create an object such as follows:

```
String component("Red LED");
```

After that, you can manipulate the string. Chapter 6 will discuss some functions for processing strings.